ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

# Ingeniería Informática + ADE

Universidad de Granada (UGR)

**Autor:** Ismael Sallami Moreno

**Asignatura:** Relación de Problemas Tema 3 : Herencia (PDOO)

# Índice

**1. Ejercicio 1** **3**
1.1. Enunciado . . . . . 3
1.2. Solución . . . . . 3

**2. Ejercicio 2** **3**
2.1. Enunciado . . . . . 3
2.2. Solución . . . . . 4

**3. Ejercicio 3** **5**
3.1. Enunciado . . . . . 5
3.2. Solución . . . . . 5

**4. Ejercicio 4** **6**
4.1. Enunciado . . . . . 6
4.2. Solución . . . . . 6
4.3. Enunciado . . . . . 6
4.4. Solución . . . . . 6

**5. Ejercicio 6** **8**
5.1. Enunciado . . . . . 8
5.2. Solución . . . . . 8

# 1 Ejercicio 1

## 1.1. Enunciado

Dada la siguiente clase en Java:

```
public class Vertebrado
{
    public ArrayList<String> partesDelAbdomen();
    public void desplazarse();
    public String comunicarse(Vertebrado vertebrado);
    protected Vertebrado obtenerCopia();
}
```

Indica con una cruz en la casilla correspondiente según si la declaración de los siguientes métodos de la clase `Mamifero`, subclase de `Vertebrado`, sobrecargan (*overloading*) o redefinen (*overriding*) a los de la clase `Vertebrado`.

## 1.2. Solución

| **Método** | **Sobrecarga** | **Redefinición** |
|---|---|---|
| public ArrayList<String>partesDelAbdomen() | | X |
| public void desplazarse(Modo m) | X | |
| public String comunicarse(Vertebrado vertebrado) | | X |
| public String comunicarse(Mamifero mamifero) | X | |
| public Vertebrado obtenerCopia() | | X |
| protected Mamiero obtenerCopia() | | X |

*Nota:Al cambiar únicamente la visibilidad o el tipo hace que sea una redefición (Últimos dos casos)*

# 2 Ejercicio 2

## 2.1. Enunciado

A partir de las siguientes clases:

```
public class Persona {
    protected String nombre;
    public Persona(String nom) { this.setNombre(nom); }
    protected String getNombre() { return this.nombre; }
    public String hablar() { return "bla bla bla"; }
}
```

```
public class Estudiante extends Persona {
    public String carrera;
    public int curso;
    public Estudiante(String nom, String carr, int cur) {
        super(nom);
```

```
        carrera = carr;
        curso = cur;
    }
    public void estudiar() { System.out.println("estudiando"); }
}
```

Implementa la clase `EstudianteInformatica` que hereda de `Estudiante`. Debe tener:

- Una nueva variable de instancia que es una colección de `String` con los dispositivos que utiliza (por ejemplo, `PC`, `tablet`, `smartphone`).
- Métodos para consultar esa variable.
- Una redefinición del método `estudiar()` para que además de estudiar como los otros alumnos, diga que estudia con el último dispositivo de su colección.
- Un constructor para inicializar la clase.

Implementa este problema en Java y Ruby:

## 2.2. Solución

```
    public class EstudianteInformatica extends Estudiante {
        private ArrayList<String> dispositivos;
        public EstudianteInformatica(string nombre, string curso){
            super(nombre, "Informatica", curso);
            dispositivos = new ArrayList<String>();
        }

        @Override
        public void Estudiar(){
            System.out.println("Estudiando Informatica con " + dispositivos
                .get(dispositivos.size() - 1));
        }
    } // Fin de la clase EstudianteInformatica
```

Listing 1: Clase Estudiante de Informática

```
    def EstudianteInformatica < Estudiante
        def initialize (nombre, curso)
            super(nombre, "Informatica", curso)
            @dispositivos = [] # @dispositivos = Array.new
        end

        def Estudiar
            super.Estudiar
            puts "Estudiando Informatica con #{@dispositivos[-1]}"
        end
    end
```

Listing 2: Clase Estudiante de Informática

# 3 Ejercicio 3

## 3.1. Enunciado

*Nota: En las diapositivas de la Relación hay un error, donde pone 3, no es el ejercicio 3 es la parte 2 del ejercicio 2, es decir, otra clase, el ejercicio 3 es el marcado con el número 4.*

Dada la siguiente clase abstracta en Java:

```java
abstract class Transporte {
    protected String marca;

    protected String getMarca() {
        return this.marca;
    }

    protected void setMarca(String marc) {
        this.marca = marc;
    }

    public abstract String hacerRuta(String origen, String destino);
}
```

Crea dos nuevas clases, `Bicicleta` y `NaveEspacial`, que hereden de `Transporte` e implementen el método `hacerRuta` de forma diferente:

## 3.2. Solución

```java
    public class Bicicleta extends Transporte {
        @Override
        public String hacerRuta(String origen, String destino) {
            return "Pedaleando desde " + origen + " hasta " + destino;
        }
    }

    def NaveEspacial < Transporte
        def hacerRuta(origen, destino)
            "Volando desde #{origen} hasta #{destino} por el espacio"
        end
    end
```

Listing 3: Clase Bicicleta

## Diagrama UML

# 4 Ejercicio 4

## 4.1. Enunciado

Hacer el ejercicio anterior, pero en Ruby.

## 4.2. Solución

```ruby
    def Bicicleta < Transporte
        def hacerRuta(origen, destino)
            "Pedaleando desde #{origen} hasta #{destino}"
        end
    end

    def NaveEspacial < Transporte
        def hacerRuta(origen, destino)
            "Volando desde #{origen} hasta #{destino} por el espacio"
        end
    end
```

Listing 4: Clase Bicicleta

## 4.3. Enunciado

Implementa en Java una clase paramétrica a partir de la cual se puedan definir clases de grupos de personas con un líder, como por ejemplo grupos de música con un cantante, o empresas con un jefe. Implementa también alguna de esas clases a partir de la clase paramétrica definida.

## 4.4. Solución

A continuación, se implementa una clase paramétrica en Java para definir grupos de personas con un líder. Posteriormente, se muestra un ejemplo de uso creando un grupo de música con un cantante.

```java
// Clase paramétrica genérica
public class Grupo<T> {
    private T lider;
    private List<T> miembros;

    public Grupo(T lider) {
        this.lider = lider;
        this.miembros = new ArrayList<>();
    }

    public T getLider() {
        return lider;
    }

    public void setLider(T lider) {
        this.lider = lider;
```

```java
    }

    public void agregarMiembro(T miembro) {
        this.miembros.add(miembro);
    }

    public List<T> getMiembros() {
        return miembros;
    }

    @Override
    public String toString() {
        return "Líder: " + lider + ", Miembros: " + miembros;
    }
}

// Ejemplo de uso con un grupo de música
public class Musico {
    private String nombre;

    public Musico(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return nombre;
    }
}

public class Main {
    public static void main(String[] args) {
        Musico cantante = new Musico("Freddie Mercury");
        Grupo<Musico> banda = new Grupo<>(cantante);

        banda.agregarMiembro(new Musico("Brian May"));
        banda.agregarMiembro(new Musico("Roger Taylor"));
        banda.agregarMiembro(new Musico("John Deacon"));

        System.out.println(banda);
    }
}
```

Si queremos implementar esta misma solución en Ruby, sería como sigue:

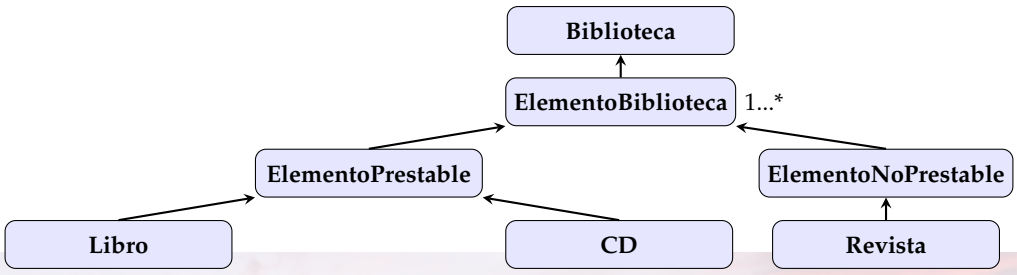
```ruby
# Clase paramétrica genérica
class Grupo
  attr_accessor :lider, :miembros

  def initialize(lider)
    @lider = lider
    @miembros = []
  end

  def agregar_miembro(miembro)
    @miembros << miembro
```

```
  end

  def to_s
    "Líder: #{@lider}, Miembros: #{@miembros.join(', ')}"
  end
end

# Ejemplo de uso con un grupo de música
class Musico
  attr_accessor :nombre

  def initialize(nombre)
    @nombre = nombre
  end

  def to_s
    @nombre
  end
end

# Crear grupo de música
cantante = Musico.new("Freddie Mercury")
banda = Grupo.new(cantante)

banda.agregar_miembro(Musico.new("Brian May"))
banda.agregar_miembro(Musico.new("Roger Taylor"))
banda.agregar_miembro(Musico.new("John Deacon"))

puts banda
```

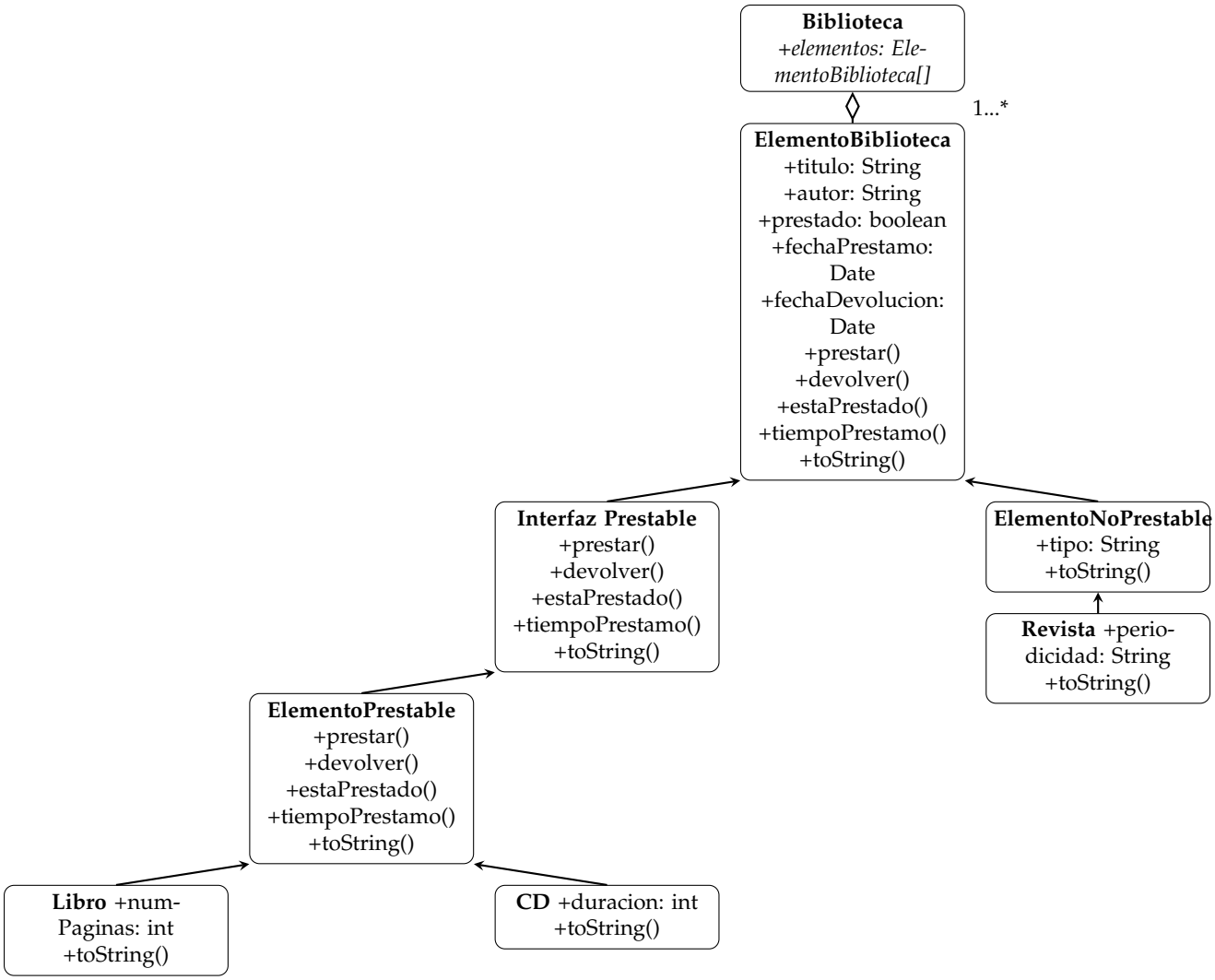
# 5 Ejercicio 6

## 5.1. Enunciado

Partiendo del siguiente diagrama de clases, modifícalo añadiendo una interfaz Prestable implementada por la clase ElementoPrestable y sus subclases. Añade a este diagrama de clases los atributos y operaciones de las distintas clases y de la interfaz Prestable.




## 5.2. Solución


Biblioteca
+elementos: ElementoBiblioteca[]
1...*
ElementoBiblioteca
+titulo: String
+autor: String
+prestado: boolean
+fechaPrestamo: Date
+fechaDevolucion: Date
+prestar()
+devolver()
+estaPrestado()
+tiempoPrestamo()
+toString()
Interfaz Prestable
+prestar()
+devolver()
+estaPrestado()
+tiempoPrestamo()
+toString()
ElementoNoPrestable
+tipo: String
+toString()
Revista +periodicidad: String
+toString()
ElementoPrestable
+prestar()
+devolver()
+estaPrestado()
+tiempoPrestamo()
+toString()
Libro +numPaginas: int
+toString()
CD +duracion: int
+toString()